

roberdan-os: sistema operativo agentico personale, cross-platform, con memoria local-first e meta-loop auto-migliorante

Roberto D'Angelo — Fight the Stroke Foundation · in coppia uomo-agente con Claude (Anthropic)

1 luglio 2026

Indice

0.1	Abstract	1
0.2	1. Introduzione	2
0.2.1	1.1 Il problema	2
0.2.2	1.2 Contributi	2
0.2.3	1.3 La metafora: Remì	3
0.3	2. Background e lavori correlati	3
0.4	3. Architettura del sistema	3
0.5	4. Canone comportamentale cross-platform	4
0.6	5. Memoria durevole local-first	4
0.6.1	5.1 Perché nel vault, non nel silo dello strumento	4
0.6.2	5.2 Il recall rotto, la causa vera, e un tentativo fallito (caso onesto)	4
0.7	6. Il meta-loop auto-migliorante	5
0.8	7. Strato di discovery: quali problemi valgono	6
0.9	8. Strato cognitivo e agenti	6
0.10	9. Valutazione	7
0.11	10. Limitazioni	8
0.12	11. Discussione: principi che generalizzano	8
0.13	12. Lavori futuri	8
0.14	13. Conclusione	9
0.14.1	Riproducibilità e artefatti	9

0.1 Abstract

Gli assistenti di codice basati su LLM (Claude Code, GitHub Copilot, OpenAI Codex) stanno diventando lo strato primario di interazione con il computer per molti knowledge worker. Ma la loro *configurazione comportamentale* — agenti, skill, hook, persona, memoria — tende a proliferare in modo frammentato: copie divergenti su decine di repo, in formati incompatibili, senza una fonte di verità unica, senza memoria portabile tra strumenti, e senza alcun meccanismo di miglioramento continuo. Presentiamo **roberdan-os**, un sistema operativo agentico personale che affronta tre problemi: (1) *frammentazione comportamentale* — un canone unico in Markdown (AGENTS.md) da cui si generano wrapper per-piattaforma; (2) *memoria siloizzata* — una memoria durevole **local-first** nel vault Obsidian dell'utente, tipata e indicizzata semanticamente, consumabile da qualunque

strumento; (3) *assenza di auto-miglioramento* — un **meta-loop** che cattura apprendimenti, li consolida sotto gate umano, e sorveglia settimanalmente le novità degli strumenti proponendo (mai applicando) adattamenti. Documentiamo inoltre uno strato di *discovery* che sposta il sistema dal “risolvere problemi” al “capire quali problemi valgono”. Riportiamo un caso empirico completo — ipotesi errata, audit, diagnosi, fix — sulla migrazione a un embedding **locale**. Il recall semantico in italiano falliva (o risultati); la prima “soluzione” (migrare a bge-m3 locale) sembrava funzionare ma un audit interno ha rivelato che il modello locale **non era attivo** e il miglioramento veniva da un banale ri-allineamento di modello. La causa del blocco era una **limitazione risolvibile del motore** (gbrain non riconosceva la dimensione nativa di bge-m3); una patch mirata di due righe alla sua *recipe* Ollama ha reso l’embedding **davvero locale** — on-device, gratuito, senza chiave API, su GPU. Il sistema è, per costruzione, **auto-proponente, mai auto-applicante** sul comportamento: la metafora guida è *Remi*, il topo di *Ratatouille* che guida lo chef — un’intelligenza che suggerisce dall’interno, mentre la mano resta umana.

0.2 1. Introduzione

0.2.1 1.1 Il problema

Un utente esperto accumula rapidamente, attraverso più strumenti agentici, un ecosistema di configurazioni: nel caso studio, ~200 skill e ~300 agenti sparsi su 13+ repository, in 3+ formati, con lo stesso agente esistente in 4-6 copie divergenti e la configurazione globale nemmeno versionata. Il problema non è **scarsità** ma **superficie di manutenzione contro uso reale**: si mantengono decine di agenti, se ne usano una manciata. Tre patologie ricorrenti:

1. **Frammentazione comportamentale.** Nessuna fonte di verità unica; il comportamento “come lavoro io” è duplicato e diverge silenziosamente tra strumenti.
2. **Memoria siloizzata.** Ogni strumento ha la sua memoria (es. la cartella memory/ di Claude Code), illeggibile dagli altri. Passando da Claude a Copilot a Codex, l’assistente “dimentica” chi sei e cosa avete deciso.
3. **Nessun miglioramento continuo.** Il sistema non impara dalle interazioni, non si riorganizza, non si aggiorna quando gli strumenti sottostanti evolvono ogni settimana.

0.2.2 1.2 Contributi

- Un **canone comportamentale cross-platform** (Sezione 4): logica in Markdown una volta sola, wrapper generati per Claude/Copilot/Codex/ChatGPT.
- Una **memoria durevole local-first** (Sezione 5): il vault Obsidian come fonte di verità tipata + indice semantico locale, con un risultato empirico sulla scelta del modello di embedding.
- Un **meta-loop auto-migliorante gated** (Sezione 6): cattura -> distillazione -> consolidamento -> sorveglianza degli strumenti, con invarianti di sicurezza enforced meccanicamente.
- Uno **strato di discovery** (Sezione 7): premortem, focus-group simulato, e un orchestratore di *problem-validation* che stima quali problemi valga la pena risolvere.
- Un **principio di design** — *auto-proponente, mai auto-applicante* — e la sua giustificazione (Sezione 10).

0.2.3 1.3 La metafora: Remi

Ratatouille (Pixar, 2007): un topo con talento culinario guida un cuoco maldestro tirandogli i capelli dall'interno del cappello. Il topo ha il gusto; la mano che cucina resta umana. roberdan-os è concepito così: un'intelligenza agentica che **suggerisce, ricorda, critica e propone** dall'interno del flusso di lavoro, mentre le decisioni irreversibili e la voce pubblica restano dell'umano. Non un pilota automatico, ma un secondo cervello con le mani legate ai gate giusti.

0.3 2. Background e lavori correlati

- **Premortem / prospective hindsight.** Klein (HBR 2007) mostra che immaginare un fallimento *già avvenuto* genera cause più specifiche e oneste che chiedere “cosa può andare storto”. Kahneman lo indicava come la sua tecnica decisionale più preziosa. È la base dello skill premortem (Sezione 7), qui potenziato da un fan-out multi-agente.
 - **Memoria a tre strati (Karpthy).** Il vault segue il pattern “sources immutabili -> wiki curata -> indice”. roberdan-os vi aggiunge uno strato di *agent-learning* tipato.
 - **Retrieval semantico ed embedding.** Il recall si appoggia a embedding densi + ricerca vettoriale (HNSW, cosine). Documentiamo (Sezione 5) come la scelta *hosted vs locale* del modello di embedding sia decisiva per costo, privacy e qualità multilingue.
 - **Digital twin & agent persona.** Lo strato *twin* modella voce e giudizio dell'utente; si distingue dai twin puramente conversazionali per l'accoppiamento con gate umani e memoria durevole.
 - **Sistemi multi-agente & reflection.** Il meta-loop e i deep-dive paralleli (premortem, focus-group) usano fan-out di agenti indipendenti + consolidamento — un pattern di reflection/ensemble applicato a decisioni personali, non solo a codice.
-

0.4 3. Architettura del sistema

Principio cardine: *conoscenza centralizzata, esecuzione per-piattaforma, comportamento unificato*. La logica vive una volta sola nel canone; i runtime la consumano tramite wrapper thin.

```
CANONE (Markdown, fonte di verità)
AGENTS.md - behavior/ - agents/ -
rules/ - skills/ - loop/ - memory/
```

```
bin/sync.sh genera consumato da
```

```
Claude Code      Copilot      Codex      ChatGPT      (altri)
```

```
runtime esegue in roberto-mode + loop + twin
```

```
MEMORIA LOCAL-FIRST (vault Obsidian + gbrain locale)      cross-platform
```

META-LOOP (launchd): capture -> distill -> curate -> evolve
auto-proponente

Tre proprietà trasversali: **daemon-optional** (nessun servizio always-on richiesto), **evidence-first** (done = artefatti verificati), **gate umani** (le azioni irreversibili passano dall'utente).

0.5 4. Canone comportamentale cross-platform

AGENTS.md è lo standard universale (letto nativamente da Codex, Copilot, Cursor); CLAUDE.md e copilot-instructions.md diventano puntatori thin. Il comportamento operativo ("come lavoro") è distillato in behavior/roberto-mode.md (autonomia, evidence-first, commit per fase, done-gate); la voce e il giudizio relazionale in behavior/roberto-voice.md; il toolkit cognitivo (first-principles, Feynman, framework decisionali) in behavior/thinking-toolkit.md. Otto agenti specializzati (architetto, security, review, done-gate, first-principles, red-team, twin, orchestratore del loop) si attivano *al momento giusto* invece di essere invocati a mano. bin/sync.sh genera i wrapper per piattaforma da questo canone unico, eliminando la divergenza.

0.6 5. Memoria durevole local-first

0.6.1 5.1 Perché nel vault, non nel silo dello strumento

La memoria durevole **non** vive nella cartella per-tool di Claude (silo illeggibile da Copilot/Codex), ma nel **vault Obsidian** dell'utente: Markdown leggibile da qualunque agente, già dotato di **ontologia tipata** (relazioni belongs_to/has/related_to), versionato, backupato, e indicizzato semanticamente da un motore locale (gbrain, Postgres + pgvector). Gli apprendimenti-agente vivono in un namespace separato (agent-learnings/, type: agent-learning) per non inquinare le note umane pur restando nello stesso grafo e recuperabili con lo stesso motore. Il silo per-tool degrada a *cache*; i suoi contenuti sono migrati nel vault.

0.6.2 5.2 Il recall rotto, la causa vera, e un tentativo fallito (caso onesto)

Durante la costruzione il recall semantico in italiano falliva silenziosamente (query -> 0 risultati) mentre in inglese funzionava. È istruttivo *come* ci siamo sbagliati sulla causa.

Prima ipotesi (parzialmente errata): il provider configurato (openai:text-embedding-3-large) aveva quota esaurita e i vettori erano prodotti da un modello debole sull'italiano; la soluzione proposta — un modello **locale multilingue** (bge-m3, 1024-dim, via Ollama) — sarebbe stata gratis, senza rate-limit, privata e forte sull'italiano. Abbiamo eseguito la migrazione: config, ALTER della colonna (1536 -> 1024), ricreazione dell'indice HNSW, re-embedding completo. Il recall italiano è passato da **0 a >= 3 risultati**. Sembrava un successo.

Cosa ha rivelato l'audit interno. Un agente di verifica empirica ha misurato lo stato reale del database e trovato che **il 100% dei chunk era ancora etichettato zeroentropyai:zembed-1**, non bge-m3. Verifica successiva: gbrain **fissa il modello di embedding all'inizializzazione** del brain (DEFAULT_EMBEDDING_MODEL = 'zeroentropyai:zembed-1'); né la modifica del config file-plane né la variabile GBRAIN_EMBEDDING_MODEL cambiano il modello a embed-time. bge-m3 è stato scaricato ma **mai usato**. Il modello locale non era attivo.

La causa vera del fix. Il recall non è migliorato per bge-m3, ma per **allineamento di modello**: prima, la *query* veniva embeddata con un modello (openai) diverso da quello dei *vettori stoccati* (zembed-1) -> spazi vettoriali incompatibili -> o risultati. Il re-embedding completo ha reso query e storage **lo stesso modello** (zembed-1@1024) -> match -> risultati. Un bug di coerenza, non di capacità linguistica.

La diagnosi della radice. L’audit ha spinto una diagnosi più profonda: la *recipe* Ollama di gbrain elencava solo nomic-embed-text (768-dim) e derivati, con default_dims: 768. Il modello scelto, bge-m3, non era in lista e la sua dimensione nativa (1024) non era nota al motore, che ripiegava sul default globale (1280) e **rifiutava** l’embed per incoerenza dimensionale. Non un limite fondamentale: **un valore mancante in un registro**.

Il fix e l’esito. Due righe nella recipe (ollama.ts: aggiungere bge-m3, portare default_dims a 1024) + ricompilazione del motore. Verifica: ollama ps mostra bge-m3 caricato al 100% su GPU e l’embed procede **senza alcuna chiave API**. L’embedding è ora davvero **locale**:

Dimensione	Prima (hosted)	Dopo il fix (locale)
Modello	zembed-1 (ZeroEntropy, hosted)	ollama:bge-m3 (on-device, GPU)
Privacy	i dati escono	restano on-device
Costo / chiave API	a consumo / richiesta	zero / nessuna
Multilingue (IT)	debole	forte (bge-m3)

Lezioni. (1) Un miglioramento osservato non prova il meccanismo ipotizzato — misurare, non dedurre. (2) Un audit avversariale interno che *misura* è ciò che ha smascherato il claim falso e condotto alla radice vera. (3) A volte l’ostacolo a una scelta architettonica corretta è un difetto *risolvibile* di una dipendenza, non un vincolo: vale la pena diagnosticarlo fino in fondo prima di rinunciare. (*Nota di stato: al momento della stesura il re-embedding locale dell’intero corpus è in corso; la qualità del recall multilingue post-migrazione va confermata a completamento.*)

o.7 6. Il meta-loop auto-migliorante

Quattro componenti, orchestrati da launchd (scattano anche a strumento chiuso), con un principio non negoziabile: **auto-proponente, mai auto-applicante** sul comportamento.

Componente	Funzione	Gate
capture	appende segnali di apprendimento a una <i>staging inbox</i> (no lock, per-piattaforma)	privacy deny-list come codice
distill	batch: classifica (tassonomia a 5 classi), dedup contro il vault, -> <i>quarantena</i>	mai scrittura diretta nel vault

Componente	Funzione	Gate
ontology/curate	<i>single-writer</i> : promuove i candidati approvati a note tipate nel vault; propone igiene (dedup, tombstone)	promozione solo se approved: true; merge/delete umani
evolve	sorveglia settimanalmente i changelog di Claude/Copilot/Codex; propone draft con citazione della fonte	mai auto-commit su behavior/ rules/ agents/ AGENTS.md

Perché non un'ontologia auto-aggiornante. Una scelta di design deliberata (validata da un'analisi first-principles avversariale) è stata **tagliare** l'idea di un'ontologia che si fonde e si riscrive da sola: si sarebbe appoggiata a un retrieval allora rotto, sarebbe stata un quarto store divergente, e l'auto-merge è lossy e irreversibile (viola il gate umano). Il 90% del valore si ottiene con **un tipo** (agent-learning) + **un job d'igiene human-gated** che *riusa* l'ontologia già esistente del vault. La struttura vive nella **cura**, non nell'**automazione del giudizio**.

0.8 7. Strato di discovery: quali problemi valgono

Il sistema non deve solo risolvere problemi, ma stimare **quali valga la pena risolvere**. Tre skill che si compongono e si auto-attivano su trigger linguistici:

- **premortem** — dato un piano, assume che sia *già fallito tra 6 mesi* e lancia **un agente per ogni causa di fallimento, in parallelo**, ognuno con storia, assunzione sottostante ed early-warning; sintetizza il fallimento più probabile, il più pericoloso, l'assunzione nascosta, il piano rivisto e una checklist. Rompe il bias di compiacenza degli LLM.
- **focus-group** — simula utenti reali: un pool di **agenti-persona** (panel persistenti nel vault + generati ad-hoc) + un **moderatore** + un **consolidatore**, in quattro modi (focus group, interviste 1:1, usability test, micro-survey). Il rischio centrale — la *sycophancy* delle personas simulate — è affrontato ancorando ogni persona a frustrazioni, alternative, budget e scetticismo reali, e pesando il segnale negativo più di quello positivo.
- **problem-validation** — orchestratore: *il problema esiste?* (focus-group) -> *vale?* (rubrica severità x frequenza x raggiungibilità x fit strategico x willingness x costo-di-sbagliare) -> *la soluzione reggerebbe?* (premortem). Sfrutta gstack a valle (spec, office-hours) invece di duplicarlo. Default **bias-to-kill**: è più prezioso dire “non vale” che confermare.

0.9 8. Strato cognitivo e agenti

Oltre agli agenti operativi, il sistema modella il **giudizio**: un *digital twin* che scrive e decide nella voce dell'utente (draft-not-send per l'esterno), un red-team (@board) obbligatorio sulle decisioni importanti, un agente first-principles (@socrates) per decostruire i problemi, e un done-gate (@thor) che è l'unico autorizzato a dichiarare “done” tramite verifica empirica. Questi non sostituiscono l'utente: **alzano la mano** al momento giusto e passano la decisione.

0.10 9. Valutazione

Metodologia. Il sistema è stato valutato da un **audit interno avversariale**: due agenti indipendenti — uno di code/ecosystem-review, uno di verifica empirica che *misura* invece di fidarsi delle affermazioni — hanno esaminato repo e stato runtime. L’audit ha trovato (e abbiamo poi corretto) tre difetti reali, documentati sotto: è esso stesso parte del risultato.

Misure verificate (stato al 1 luglio 2026):

Claim	Misura reale	Esito
Recall italiano non più a zero	3/3 query IT -> 3 risultati (score 0.830.90)	[OK] (rilevanza da riconfermare post re-embed bge-m3)
Embedding locale attivo	ollama ps: bge-m3 100% GPU; embed keyless; patch gbrain f7376b11	[OK] locale ; re-embed corpus in corso
Memoria migrata al vault	19 note type: agent-learning, committate, indicizzate, recuperabili	[OK]
Meta-loop pipeline	capture -> distill -> curate testata end-to-end; launchd exit 0	[OK]
Privacy gate come codice	capture/curate bloccano i nomi della deny-list, passano il contenuto normale	[OK] dopo il fix (vedi sotto)
CI verde	test/validate.sh -> TUTTO VERDE (frontmatter, link, drift, shellcheck, leak)	[OK] dopo il fix
Superficie del canone	~700 righe Markdown per comportamento+memoria+meta-loop+discovery	[OK] (compatto by design)

Difetti trovati dall’audit e corretti nella stessa sessione: 1. **Embedding “locale” era falso** (Sezione 5.2) — riformulato, label DB ripristinati alla verità. 2. **CRITICAL — privacy non era “codice”**: capture/distill filtravano solo la stringa letterale del path, non il contenuto; curate non aveva alcun check. *Fix*: deny-list reale (private/.denylist) matchata prima di ogni write, con esclusione delle righe vuote/commento. Testato. 3. **HIGH — il repo non passava il proprio CI**: il commit degli skill non aveva rigenerato i wrapper platforms/. *Fix*: wrapper generati e committati; validate.sh ora verde. 4. **MEDIUM — mismatch doc/impl**: ADR/evolve citavano validate.sh per la path-allowlist; l’enforcement reale è in post-task-sync.sh (git add scoped). Corretto.

0.11 10. Limitazioni

- **Embedding locale appena raggiunto, non ancora consolidato.** Dopo la patch (Sezione 5.2) l'embedder è locale (bge-m3 su GPU), ma il re-embedding dell'intero corpus (~51k chunk) è in corso al momento della stesura; la qualità del recall multilingue va misurata a completamento. La patch vive in un fork locale di gbrain e va ri-applicata dopo un update del motore.
 - **Costruito != attivo (in parte sanato).** In questa sessione il meta-loop è stato *attivato* (launchd caricati, hook wired, memoria migrata), ma le pipeline sono in **bootstrap** (inbox e quarantena con pochi elementi), non a regime; le metriche longitudinali mancano.
 - **Seam di giudizio dichiarati.** distill scrive class: TODO (nessuna classificazione automatica) e evolve fa solo fingerprint-diff dei changelog: entrambi richiedono un agente-nel-loop. Il design **rende il gap innocuo** (curate rifiuta i TODO; evolve è draft-only), ma il giudizio non è automatizzato — e non deve esserlo.
 - **La sicurezza va verificata, non assunta.** L'audit ha mostrato che l'invariante privacy era una *promessa testuale + gate umano*, non codice, finché non l'abbiamo implementata. Ciò illustra il rischio generale: le invarianti di un sistema auto-referenziale vanno **testate**, perché la documentazione può divergere dall'implementazione.
 - **Sycophancy residua.** Il focus-group mitiga ma non elimina la compiacenza delle personas simulate; è uno strumento di scoperta di domande e frizioni, non un sostituto di utenti reali.
 - **Single-user, single-machine.** Calibrato su un individuo e una macchina; la portabilità cross-platform è progettata ma verificata primariamente su Claude Code.
 - **Rischio di auto-modifica.** Un sistema che propone modifiche a se stesso richiede invarianti meccaniche (draft-only, git-add scoped, gate umani, deny-list); la loro robustezza è un'assunzione da sorvegliare continuamente, non un fatto acquisito.
-

0.12 11. Discussione: principi che generalizzano

1. **Local-first per la memoria personale.** Privacy, costo-zero e disponibilità superano la marginale qualità in più di un modello hosted — specie in contesti multilingue e sensibili.
 2. **Auto-proponente, mai auto-applicante.** L'autonomia utile si ferma prima dell'irreversibile; il valore sta nel *proporre con evidenza*, non nell'agire da solo sul comportamento.
 3. **Riuso > reinvenzione.** La tentazione di costruire nuova struttura (un'ontologia, un motore) spesso aggiunge superficie di manutenzione senza ROI; estendere ciò che esiste è quasi sempre meglio.
 4. **Discovery prima di solving.** Il valore più alto non è risolvere meglio, ma scegliere il problema giusto — e questo richiede la voce dell'utente e lo stress-test del fallimento *prima* del build.
-

0.13 12. Lavori futuri

- **Consolidare l'embedding locale:** completare il re-embed bge-m3 dell'intero corpus, misurare la qualità del recall multilingue, e proporre la patch upstream a gbrain (registrare bge-m3@1024) così non serva un fork locale.
- Verifica della portabilità reale su Copilot e Codex (non solo progettata).
- Capture automatico a partire dai transcript di sessione (oggi agent-driven).

- Panel di focus-group calibrati su audience reali con consenso.
- Metriche longitudinali del meta-loop (quanti apprendimenti promossi sopravvivono alla revisione).

0.14 13. Conclusione

roberdan-os mostra che è possibile dare a un individuo un *sistema operativo agentico* coerente tra strumenti, con memoria propria, privata e persistente, e un ciclo di miglioramento che non tradisce mai il controllo umano. Non un assistente in più, ma un secondo cervello con le mani ai gate giusti — Remì nel cappello, con il gusto giusto e la mano che resta umana.

0.14.1 Riproducibilità e artefatti

Codice e canone: repository git roberdan-os (~15 commit al 1 luglio 2026). Memoria: vault Obsidian (local-first) + gbrain (Postgres/pgvector locale); embedding **locale** ollama:bge-m3@1024 (via patch alla recipe Ollama di gbrain, commit f7376b11; re-embed del corpus in corso alla stesura). Scheduling: launchd. Canone, memoria, orchestrazione e embedding sono ora local-first — nessun servizio cloud né chiave API per il funzionamento core. Verifiche empiriche e audit riproducibili via `test/validate.sh`, `ollama ps`, e `query gbrain`.